Redis Caching Reference (Days 56-57)

Goal: cache Account reads, invalidate correctly on every write. The risk isn't the Redis mechanics — it's forgetting an eviction and silently serving a stale balance.

1. Dependency (pom.xml)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

2. Run Redis locally

```
docker run -d --name redis -p 6379:6379 redis:7
```

3. application.properties

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.cache.type=redis
spring.cache.redis.time-to-live=600000
# 10 minutes TTL — never cache forever; a bug elsewhere shouldn't mean a
# wrong balance is served indefinitely.
```

4. Enable caching

On `EnterpriseDigitalBankingSystemApplication.java`:

```
@SpringBootApplication
@EnableCaching
public class EnterpriseDigitalBankingSystemApplication { ... }
```

5. Cache the reads

In `AccountServiceImpl`:

```
@Override
@Transactional(readOnly = true)
@Cacheable(value = "accounts", key = "#accountId")
public AccountResponse getAccountById(Long accountId) {
    return accountMapper.toResponse(findAccountById(accountId));
}

@Override
@Transactional(readOnly = true)
@Cacheable(value = "accounts", key = "#accountNumber")
public AccountResponse getAccountByAccountNumber(String accountNumber) {
    ...
}
```

6. Evict on every write path — this is the part that's easy to miss

Every method that changes balance or status needs `@CacheEvict`. Because the cache has two different keys for the same account (`accountId` and `accountNumber`), evicting by one key alone leaves the other stale — the simplest correct option is `allEntries = true` on the `"accounts"` cache for every mutation:

```
@Override
@CacheEvict(value = "accounts", allEntries = true)
public AccountResponse updateAccount(...) { ... }

@Override
@CacheEvict(value = "accounts", allEntries = true)
public AccountResponse freezeAccount(...) { ... }

@Override
@CacheEvict(value = "accounts", allEntries = true)
public AccountResponse closeAccount(...) { ... }
```

Also evict from `TransactionServiceImpl` — `deposit()`, `withdraw()`, and `transfer()` all change balances on `Account` rows directly, but live in a different service. Easiest fix: inject a reference to the cache (`CacheManager`) or — simpler — call a small helper in `AccountService` to evict after each transaction, or just annotate those three methods with the same `@CacheEvict(value = "accounts", allEntries = true)` even though they're not in `AccountServiceImpl`. Spring's cache eviction works on any bean method, not just the one that originally cached the data.

Testing instructions (do this yourself, no app server needed beyond your own)

1. Start Redis (`docker run ...` above) and the banking app.
2. `GET /api/v1/accounts/{id}` twice in a row. Watch the app's console: `spring.jpa.show-sql=true` is already on — the **first** call should print a `select ... from accounts` SQL statement, the **second** should print **nothing** (served from cache).
3. Open a second terminal: `docker exec -it redis redis-cli`, then `KEYS *` — you should see a key like `accounts::1`. `GET` it to see the cached JSON.

4. Now `POST /api/v1/transactions/deposit` on that same account, then `GET /api/v1/accounts/{id}` again — confirm the **new** balance comes back (not the stale cached one) and that the SQL log shows a fresh query (proves the evict worked). If you see the *old* balance, you missed an eviction somewhere in the write path above.